

# CRISISMESH: DECENTRALIZED P2P MESH NETWORK WITH ON-DEVICE EDGE AI

Project Architecture, Engineering Process, Resource Utilization & Technical Challenges Report

|                                                         |                                                                |
|---------------------------------------------------------|----------------------------------------------------------------|
| <b>Project:</b> CrisisMesh                              | <b>Target Hardware:</b> Qualcomm Snapdragon (Nothing Phone 3a) |
| <b>Core Model:</b> Google Gemma 3 270M (INT8 Quantized) | <b>Engine:</b> MediaPipe LiteRT LlmInference Native Bridge     |
| <b>Network Protocol:</b> Wi-Fi Direct & BLE P2P Mesh    | <b>Status:</b> Fully Functional (Scriptless On-Device AI)      |

## 1. Executive Summary & Vision

CrisisMesh is a mission-critical emergency triage and communication platform built to operate in total communication blackouts. During natural disasters (hurricanes, earthquakes, floods) or infrastructure grid failures, centralized cellular towers and internet backbones fail. CrisisMesh solves this dual crisis by pairing a **peer-to-peer (P2P) mesh network** (for decentralized signal relay) with **true on-device AI** (Google Gemma 3 270M) running locally on edge hardware NPU chips. Emergency medical triage, victim condition parsing, and rescue routing occur entirely offline without transmitting data to cloud servers.

## 2. Problem Statement

The system directly addresses three critical bottlenecks in conventional disaster response:

- Single Point of Failure in Connectivity:** Traditional emergency systems (e.g., 911, cloud messaging) become completely inaccessible when cell towers collapse.
- Triage Overhead & Decision Bottlenecks:** In mass-casualty events, first responders are overwhelmed. Without automated triage, critical patients wait alongside minor injury victims.
- Cloud AI Incompatibility:** Standard LLM solutions (GPT-4, Claude) rely on high-bandwidth APIs. Existing attempts to bring AI to edge devices frequently crash due to memory leaks, buffer overflows, or excessive thermal throttling.

## 3. Solution Reached & System Architecture

The finalized CrisisMesh architecture eliminates reliance on scripted decision trees or remote cloud servers. The application integrates a low-latency, hardware-accelerated **Gemma 3 270M** quantized model directly into the Android native layer.

| Component           | Technology Stack & Implementation Details                                                      |
|---------------------|------------------------------------------------------------------------------------------------|
| User Interface      | Flutter 3.x with Riverpod state management for fluid, real-time message streaming.             |
| Native Bridge       | Kotlin MainActivity.kt utilizing Flutter MethodChannel('com.crisismesh.ai').                   |
| AI Inference Engine | Google MediaPipe tasks-genai (LiteRT) loading gemma3-270m-it-q8.task directly to NPU.          |
| P2P Mesh Network    | Wi-Fi Direct & Bluetooth Low Energy (BLE) 'Dumb Relay' graph topology for message hop routing. |

## 4. Engineering Process & Milestone Progression

The development followed a rigorous, empirical phase-by-phase implementation:

### Phase 1: Legacy Codebase Purge

Completely stripped out all hardcoded decision trees (decision\_tree.dart), rule-based keyword matchers, and legacy ScriptedTriageEngine fallbacks. The chat controller was re-architected to rely exclusively on the live LiteRTGemmaEngine interface.

### Phase 2: Android Native C++ / Kotlin Bridge

Implemented asynchronous MethodChannel handlers in MainActivity.kt to manage model lifecycle. Configured automatic asset copying from `src/main/assets/gemma.task` into context.cacheDir to supply absolute file system paths to the native C++ MediaPipe binary.

### Phase 3: Hardware NPU Optimization & Model Selection

Tested multiple parameter scales. While 4B and 2B models suffered from high memory pressure on mobile devices, the **Gemma 3 270M (INT8 Quantized)** model provided the optimal balance of zero-shot reasoning, near-zero latency, and low thermal impact.

### Phase 4: Verification on Physical Hardware

Deployed the debug APK directly to a physical **Nothing Phone (3a)** powered by a Qualcomm Snapdragon 7s Gen 3 processor. Verified unscripted, dynamic AI responses to unexpected prompt inputs (e.g. stomach pain, burns, lacerations).

## 5. Key Resources & Dependencies Used

| Resource              | Description / Specification                                                      |
|-----------------------|----------------------------------------------------------------------------------|
| Physical Hardware     | Nothing Phone (3a) — Qualcomm Snapdragon 7s Gen 3 SoC (Hexagon NPU & Adreno GPU) |
| AI Model Binary       | Google Gemma 3 270M (Instruction-Tuned, 8-bit quantized: gemma3-270m-it-q8.task) |
| SDKs & Native Libs    | Flutter 3.x, Android SDK 34, com.google.mediatrace:tasks-genai:0.10.14           |
| Dev Tools & Utilities | ADB (Android Debug Bridge), Git, Android Studio, Ripgrep, PowerShell             |

## 6. Key Engineering Challenges Faced & Solutions

### Challenge 1: Native C++ Buffer Overflows & OOM Crashes on Larger Models

*Issue:* Initial attempts to run 4B and 2B models (e.g. MedGemma 4B) resulted in native SIGSEGV crashes due to memory allocation failures when compiling attention masks.

*Solution:* Swapped the model architecture to **Gemma 3 270M INT8**, reducing the memory footprint to ~960MB while preserving full instruction-following capability.

### Challenge 2: ADB Storage Bottlenecks & Insufficient Disk Space

*Issue:* Bundling 1GB+ tensor weights into local assets caused build directory bloat and triggered `INSTALL_FAILED_INSUFFICIENT_STORAGE` on test devices.

*Solution:* Implemented automated asset cleanups, cleared app cache via `adb shell pm clear`, and configured `.gitignore` to omit heavy binaries from Git tracking.`

### Challenge 3: Model Format Mismatch (.litertlm vs .task FlatBuffers)

*Issue:* Attempting to run Llama 3.2 1B (`.litertlm` format) threw Unable to open zip archive because MediaPipe LlmInference specifically requires zip-based .task` bundles.`

*Solution:* Verified binary headers using byte inspection (4C 49 54 45 52 54 4C 4D) and standardized the production build around verified Google Gemma MediaPipe `.task` bundles.`

### Challenge 4: Silent Cache Stale Model Persistence

*Issue:* Android ``cacheDir` persisted old model files across APK updates, causing the app to silently skip extracting updated models.`

*Solution:* Added explicit cache verification routines to invalidate old model binaries upon version updates.

## 7. Future Roadmap & Enhancements

- **Dynamic Suggestion Chip Extraction:** Parse LLM output formatting to extract interactive next-step chips automatically.

- **Multimodal Vision Triage:** Enable on-device image tensor evaluation for wound and burn severity assessment.
- **Cross-Platform Mesh Gateway:** Extend mesh routing protocol across iOS (MultipeerConnectivity) and desktop field stations.

---

Report Generated: August 2026 | CrisisMesh Open Source Project